An Empirical Audit of Input Encoders for Multi-Channel Signal Transformers

Ossi Lehtinen*

Ocon Oy

Abstract

Transformers consuming multi-channel scalar signals must embed C simultaneous values into one d_{model}-dimensional vector per time step. We empirically audit eight input encoders—spanning a shared-scalar baseline, per-channel linear projections, an orthogonality regulariser, a nonlinear MLP stem, block-partitioned concatenation, channel-independent and channel-as-token architectures, and a projected positional encoding—on a synthetic benchmark designed to make channel identity informative and on ETTh1 as a real-data check, measured in next-step negative log-likelihood (NLL). The headline is one of practical near-equivalence within a wide "top tier": the standard per-channel linear projection (`nn.Linear(C,` $d_{\texttt{model}}$`)`) matches every alternative in that tier up to small, statistically real but practically modest, differences. Two encoders lose decisively: the shared-scalar baseline, which collapses for information-theoretic reasons we make explicit, and the channel-independent PatchTST-spirit baseline, which underperforms on both benchmarks and overfits universally on the synthetic one. Paired tests resolve two small gaps: projecting the sinusoidal positional encoding through a learned linear layer edges the rest at small C, with a direct geometric probe showing the mechanism is positional–channel orthogonalisation; a nonlinear MLP stem edges them at the largest C we test, with the gap shrinking under more training data. The practical recommendation is to use `nn.Linear(C,` $d_{\texttt{model}}$`)` by default and reach for something more elaborate only when the task at hand gives a real reason to do so. Code and data to reproduce every experiment in this paper are available on GitHub.[1]

1 Introduction

Transformers applied to multivariate numerical signals—multi-sensor telemetry, financial factor stacks, biomedical waveforms—face a design choice at the input layer: how to embed C simultaneous scalar channels into one d_{model}-dimensional vector per time step. The central objective is to prepare the input information for the the subsequent transformer layer with minimal losses, thus allowing maximal flexibility for modeling the influence and interplay of the signals in relation to the target variables.

Many approaches have been put forward for achieving just this. The additive embedding recipe inherited from NLP (9) projects each input to a vector and sums. When the inputs are multiple channels rather than one token-plus-position pair, this comes in two forms: a *shared* scalar projection W with per-channel additive embeddings, or a *per-channel* projection W_k—the latter being mathematically a single `nn.Linear(C,` $d_{\texttt{model}}$`)`. Many multivariate time-series transformers

*`ossi@ocon.fi`. Anthropic's Claude Code and the Opus 4.6 and 4.7 models were extensively used in preparing the experiments and writing this article.

[1] https://github.com/OssiLehtinen/channel-encoder-audit

use a linear input stem (10, 14), though details vary. Alternative architectures treat each channel as a separate token (iTransformer (7)), run independent backbones per channel (PatchTST (8)), or combine both (Crossformer (13)). Orthogonality penalties on weight matrices have been used for feature decorrelation in deep networks (2).

Despite the variety of approaches, several basic questions remain unanswered in the literature: (i) Compared to a bare `nn.Linear(C,` $d_{\texttt{model}}$`)` projection, what do we actually gain—in held-out loss, in geometric structure, or in convergence speed—from more involved architectures, such as block partitioning, an orthogonality penalty, a nonlinear MLP stem, or a channel-independent / channel-as-token architecture? (ii) Does the task loss itself produce per-channel separation without explicit enforcement? (iii) Does the positional encoding interact with the channel encoding, and can this interaction be improved? We include a shared-scalar baseline (SUM) as a verified floor rather than an open question—the encoder algebraically collapses C channels to their pointwise sum, so its failure mode is information-theoretic and the only quantity of interest is how much downstream capacity fails to recover from it.

Here, we compare eight encoders on a controlled synthetic benchmark designed to make channel identity informative, with supporting experiments on the public ETTh1 dataset (14). Three answers map onto the three open questions above. First, the per-channel linear projection matches every alternative at convergence up to small, statistically real but practically modest differences—the encoder landscape inside the per-channel-W_k family is one of practical near-equivalence at 20 paired seeds. Second, the task loss alone drives the learned W_k toward near-orthogonality and scales their norms with channel importance, with no explicit regulariser required. Third, a learned linear projection of the sinusoidal positional encoding gives a small but consistent gain, and a direct geometric probe shows the mechanism is positional–channel orthogonalisation rather than positional subspace compression. The shared-scalar SUM baseline, included as a verified floor, hits the capacity-independent ceiling its information-theoretic derivation predicts. Two narrow exceptions to the practical-near-equivalence verdict survive paired analysis but stay small in magnitude: MLP edges the linear family at C=16 with the gap shrinking ∼3× under 10× training data, and CAT posts the lowest mean NLL on ETTh1 but is statistically tied with LINEAR and LINEAR-ORTHO.

2 Method

2.1 Model architecture

Six of the eight encoders (SUM, LINEAR, LINEAR-ORTHO, MLP, LINEAR-PPE, CONCAT) share a common backbone: a small causal transformer with d_{model}=64, 4 attention heads, 3 layers, feed-forward width d_{ff}=256, dropout 0.1, pre-LayerNorm, and a linear categorical head mapping to K=32 logits. For these six, the *only* component that varies is the input encoder (Figure 1). The other two encoders (CI and CAT) are architectural alternatives that reshape the token sequence the backbone consumes; their structure is described in Section 2.2 and we still match d_{model}, head count, and layer count to the shared backbone so the architectural cost of those alternatives is comparable.

2.2 Encoders

We study eight encoders. In the definitions below, $v_k(t)$ is the scalar value of channel k at time t, $\mathbf{p}(t)$ is a fixed sinusoidal positional encoding (9), and all learned parameters are randomly initialised.

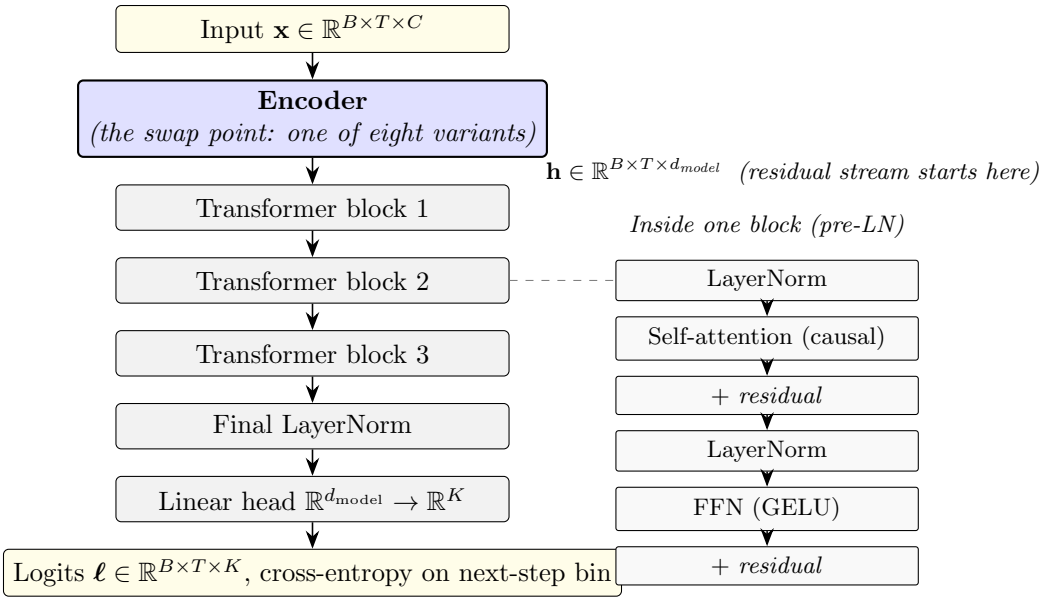

Figure 1: Architecture. The diagram shows the encoder-swap setup used by six of the eight variants (SUM, LINEAR, LINEAR-ORTHO, MLP, LINEAR-PPE, CONCAT): the same causal transformer backbone (three pre-LN blocks, each structured as in the inset on the right: LN $\to$ self-attention $\to$ residual, LN $\to$ FFN $\to$ residual; followed by a final LayerNorm and a linear head into K=32 bin logits) is used throughout, and **the only component that varies is the encoder (blue)**. These six encoders share the input/output signature $\mathbb{R}^{B\times T\times C} \to \mathbb{R}^{B\times T\times d_{\text{model}}}$ and are drop-in interchangeable. The remaining two variants—CI (channel-independent) and CAT (channel-as-token)—are full-architecture alternatives, not encoder swaps: CI runs the transformer once per channel on $(B{\cdot}C, T, d_{\text{model}})$ tensors and concatenates the per-channel hiddens at the head; CAT uses a $C{\cdot}T$-long token sequence with attention over $(B, C{\cdot}T, d_{\text{model}})$. Both are included as architectural baselines and are defined in full in Section 2.2.

sum (shared-scalar baseline). Shared scalar projection $W \in \mathbb{R}^{d_{\text{model}}}$, learned per-channel additive embedding e_k:

$$\mathbf{h}(t) = \sum_k \big(W\, v_k(t) + e_k\big) + \mathbf{p}(t).$$

Total encoder capacity is $\mathcal{O}(d_{\text{model}})$ regardless of C. We use this to isolate what goes wrong when the input projection shares weights across channels.

linear. Per-channel projection W_k and bias b_k, summed:

$$\mathbf{h}(t) = \sum_k \big(W_k\, v_k(t) + b_k\big) + \mathbf{p}(t).$$

Equivalently $\mathbf{h}(t) = W\,\mathbf{v}(t) + \mathbf{b} + \mathbf{p}(t)$ with $W \in \mathbb{R}^{d_{\text{model}}\times C}$: a single `nn.Linear(C, d_model)`.

linear-ortho. Same as LINEAR with an auxiliary loss $L_{\text{ortho}} = \lambda \sum_{i\neq j} (W_i \cdot W_j)^2/2$ (λ=10^{-2}). This variant isolates the regulariser's contribution.

linear-ppe (projected positional encoding). Same channel encoding as LINEAR, but the sinusoidal position passes through a learned linear layer:

$$\mathbf{h}(t) = \sum_k \big(W_k\, v_k(t) + b_k\big) + W_{\text{pos}}\, \mathbf{p}(t) + \mathbf{b}_{\text{pos}},$$

with $W_{\text{pos}} \in \mathbb{R}^{d_{\text{model}} \times d_{\text{model}}}$. The hypothesis is that the optimiser can then rotate the positional subspace to a channel subspace better compatible with the signal channel encodings. Adds $d_{\text{model}}^2 + d_{\text{model}}$ parameters (4,160 at d=64). The construction is closely related to the untied positional embedding of Ke et al. (5), which removes the additive position–content coupling at the input layer of language transformers via a separate projection; LINEAR-PPE applies the same decoupling principle to the channel–position competition in time-series transformers.

mlp. Two-layer MLP on the channel vector: $\mathbf{h}(t) = W_2\, \text{GELU}(W_1\, \mathbf{v}(t) + \mathbf{b}_1) + \mathbf{b}_2 + \mathbf{p}(t)$, with $W_1 \in \mathbb{R}^{d_{\text{model}} \times C}$, $W_2 \in \mathbb{R}^{d_{\text{model}} \times d_{\text{model}}}$. Tests whether nonlinearity helps ($\sim$9$\times$ more encoder parameters than LINEAR).

concat. Per-channel projection into d_{block}=d_{model}/C dims, concatenated (requires $C \mid d_{\text{model}}$): $\mathbf{h}(t) = [\mathbf{e}_0(t) \| \cdots \| \mathbf{e}_{C-1}(t)] + \mathbf{p}(t)$. Channel identity is an architectural invariant.

ci (channel-independent). PatchTST-spirit (8): shared-weight causal transformer per channel; final hidden states concatenated into the head. The computational cost of this approach is high with $\mathcal{O}(C\,B\,T^2)$ scaling.

cat (channel-as-token). iTransformer-spirit (7): each (t, k) pair is a token, sequence length $C{\cdot}T$, causal across time, bidirectional within a time step. Attention scales as $\mathcal{O}((CT)^2)$, which makes CAT substantially heavier than the other encoders at every C and prohibitively expensive at C=16, T=160 on our compute budget; we skip C=16 for that reason.

2.3 Training protocol

Identical across encoders and across both datasets unless noted.

Objective. At every position $t \in \{0, \ldots, T{-}1\}$ the model emits logits $\ell_t \in \mathbb{R}^K$ and we minimise the next-step categorical cross-entropy (negative log-likelihood, NLL)

$$\mathcal{L}_{\text{NLL}} = -\frac{1}{B(T{-}1)} \sum_{b=1}^{B} \sum_{t=0}^{T-2} \log\, \text{softmax}(\ell_t^{(b)})_{\,y_{t+1}^{(b)}},$$

where $y_t^{(b)}$ is the bin index at position t in series b. The predicted distribution at position t is supervised only by the bin at position $t+1$, and causal masking inside the transformer prevents the model from seeing y_{t+1} at prediction time. LINEAR-ORTHO adds the auxiliary orthogonality term defined in Section 2.2; all other encoders have $\mathcal{L} = \mathcal{L}_{\text{NLL}}$. As a loss-family robustness check we also re-run a subset of the synthetic sweep with a scalar regression head and MSE loss on the continuous (pre-binning) target; this is described in Section 3.11.

Optimiser. AdamW with peak learning rate 3×10^{-4}, weight decay 10^{-4}, and PyTorch defaults for the moment estimates (β_1=0.9, β_2=0.999, ϵ=10^{-8}). Gradient norm is clipped to 1.0 per step, batch size 32. The learning rate follows a cosine annealing schedule from peak to 3×10^{-6} (1% of peak) over the full 300 epochs, with no warm-up: the pre-LN architecture (11) is gradient-stable at initialisation and our convergence flags across the 20-seed sweep (Section 3.9) show no early instability on any of the top-tier encoders.

Train/val split. For the synthetic benchmark we generate n_{series}=512 length-T=160 series per seed and split with a seeded `torch.utils.data.random_split` into 461/51 ($\approx$ 90/10) train and val. A separate "data-rich" condition with n_{series}=5120 (10× training data, same 90/10 split) is used at C=16 to check whether the encoder ordering at our main-sweep scale is data-limited; this condition is reported in Section 3.2. For ETTh1 we follow the dataset's standard chronological partition, using the train block to fit per-variate standardisation statistics and the quantile bin edges for the target, and evaluating on the val block.

Validation cadence and model selection. Validation NLL and top-1 accuracy are evaluated every 20 epochs (plus epochs 1 and 300). We report each run's best validation NLL along this trace and the accuracy at the same epoch—"effective early stopping", applied uniformly. Training is never halted; every run sees the full 300-epoch schedule.

Seeds and reproducibility. Headline configurations are run over 20 random seeds, indexed 0 through 19. Seed s sets `torch.manual_seed(s)` (controlling weight initialisation and dropout) and seeds the `torch.Generator` passed to `random_split` (controlling the train/val partition); on the synthetic benchmark it also drives the `numpy` RNG that generates the input series. *The same* 20 *seeds are reused for every encoder variant, every C, and every d_{model}.* This is a paired-seed design: at a fixed s, every encoder sees identical training data and the identical train/val partition, and only the model architecture (and its initial parameter draws from the shared PRNG state) differs. We *report* unpaired mean $\pm$ std intervals throughout for readability, but we *test* with paired-difference statistics where the unpaired interval is loose enough that paired analysis changes the conclusion. Where the paired and unpaired test agree qualitatively (most rows in this paper) we report only the unpaired interval; where they disagree, or where the unpaired p-value is borderline, we report the paired result and call it out. A few diagnostics that consume substantially more compute or are by-design single-instance—linear probing, test-time channel masking, the channel-bias ablation, and the MLP first-layer geometry table—are kept at 5 seeds and labelled as such in their captions. Bootstrap confidence intervals on cosine, paired-difference, and similar derived quantities are constructed by resampling seeds (and, where applicable, examples) with 10,000 resamples; reported as 95% intervals. We do not enable `torch.use_deterministic_algorithms(True)`, so CUDA matmul nondeterminism contributes some additional seed-level variation between full re-runs; the reported $\pm$ std intervals capture this.

2.4 Datasets

Synthetic. N=512 time series of length T=160 with C channels. Each channel is a sum of three sinusoids with random frequencies in $[0.005, 0.08]$ cycles/sample plus AR(1) noise, standardised. The outcome is a fixed non-linear function of the first four channels with lags 3 and 7:

$$y_t = \tanh\big(s_0(t-3)\, s_1(t)\big) + 0.6\sin\big(1.3\, s_2(t-7)\big) + 0.4\,\mathbb{1}[s_3(t) > 0]\, s_0(t),$$

binned into K=32 quantile bins. Channels $k \geq 4$ are independent distractors. The task is designed so that an encoder that mixes channels at the input layer cannot recover the interaction structure.

ETTh1. Electricity Transformer Temperature (14), hourly, 7 variates. We standardise on the train split, bin the target variate OT into 32 quantile bins, and predict the next-step bin given all 7 variates over T=160. The model uses d_{model}=56 (divisible by C=7), 7 heads, d_{ff}=224; training is otherwise identical.

2.5 Diagnostic analyses

Beyond aggregate NLL, we use three diagnostics to inspect what the per-channel-W_k encoders actually learn. Each is computed after full 300-epoch training and aggregated over 20 seeds unless stated otherwise (the channel-bias ablation, linear probing, test-time channel masking, and the MLP first-layer geometry table use 5 seeds).

Gram-matrix analysis. LINEAR and LINEAR-ORTHO differ only in the auxiliary orthogonality penalty (Section 2.2). If both reach the same NLL, the question is whether LINEAR's W_k are geometrically similar to LINEAR-ORTHO's—i.e. whether the task pressure alone already produces a near-orthogonal arrangement—or whether the two encoders arrive at the same loss through different geometries.

After training we extract the per-channel projection matrix $W \in \mathbb{R}^{C \times d_{\text{model}}}$ (rows W_k) from the encoder and compute the cosine matrix $G_{ij} = (W_i \cdot W_j)/(\|W_i\|\|W_j\|)$. Two scalar summaries:

$$\overline{|\cos|} = \frac{1}{C(C-1)} \sum_{i \neq j} |G_{ij}|, \qquad \max|\cos| = \max_{i \neq j} |G_{ij}|.$$

Both go to zero iff the W_k become mutually orthogonal. We report seed-averaged means.

Encoding-space allocation. Orthogonality says channels are distinguishable; it does not say whether the model dedicates more representational capacity to outcome-relevant channels than to distractors. We measure two quantities, both at the encoder output:

(i) Norm $\|W_k\|$ for each channel. A larger norm means channel k's contribution $W_k v_k(t) + b_k$ has larger magnitude in the embedding, i.e. occupies more of the residual stream.

(ii) Per-channel variance fraction. The encoder output is a sum of per-channel contributions $W_k v_k(t) + b_k$. Treating each contribution as a random vector indexed by (t, batch), the per-channel variance is

$$\sigma_k^2 = \sum_{d=1}^{d_{\text{model}}} \text{Var}_{t,\text{batch}}\big[(W_k v_k(t) + b_k)_d\big],$$

and we report fractions $\sigma_k^2 / \sum_j \sigma_j^2$ on a held-out batch. This combines the size of W_k with the empirical variability of channel k's input distribution.

For both metrics, channels are partitioned into drivers ($k \in \{0, 1, 2, 3\}$, which enter the outcome formula) and distractors ($k \geq 4$, independent of the outcome). Reporting them this way lets us check whether the model independently discovered the driver/distractor split.

Table 1: Main comparison at C=4, d_{model}=64, 300 epochs, cosine LR decay, mean ± std over 20 seeds (5 canonical + 15 paired-seed extras). Best val NLL (effective early stopping). Random baseline: NLL = $\ln 32 \approx 3.47$, acc =0.031. †For CI and CAT, "encoder params" counts only the per-channel input projection (plus per-variate embedding for CAT); the bulk of those architectures' parameters lives in the replacement backbone.

Encoder	Enc. params	Val NLL ↓	Val acc ↑
SUM	320	3.257 ± 0.011	0.091 ± 0.003
CI†	128	3.053 ± 0.013	0.136 ± 0.004
CAT†	384	2.348 ± 0.060	0.212 ± 0.011
CONCAT	128	2.170 ± 0.025	0.243 ± 0.006
LINEAR	512	2.155 ± 0.019	0.247 ± 0.006
LINEAR-ORTHO	512	2.155 ± 0.020	0.247 ± 0.006
MLP	4,480	2.177 ± 0.022	0.242 ± 0.007
LINEAR-PPE	4,672	2.114 ± 0.029	0.256 ± 0.009

Linear-probing analysis. The encoder lays down a representation; the question is whether downstream attention and FFN layers preserve enough of it that each channel remains linearly identifiable several blocks deep. The pre-LayerNorm transformer's residual structure already makes a zeroth-order prediction here: each block applies $h \leftarrow h + \text{Attn}(\text{LN}(h))$ and $h \leftarrow h + \text{FFN}(\text{LN}(h))$, so an unmodified copy of the input embedding is carried forward through every block. Under the residual-stream view formalised by Elhage et al. (4), the residual stream is a communication bus that sublayers read from and additively write into; we should therefore expect deep linear probes (1) to recover whatever was written to the residual stream at layer 0, modulo what LayerNorm rescales and what the sublayers additively perturb. We test this directly by training closed-form ridge probes from a frozen hidden state back to the raw input.

Procedure: (1) train the encoder–transformer end-to-end as usual; (2) freeze it; (3) generate a fresh probe dataset (`MultiSignalDataset` with a different seed, so the model has not been trained on it); (4) for each layer of interest, pass the probe dataset through and collect hidden states $H \in \mathbb{R}^{N \times d_{\text{model}}}$ alongside the corresponding raw inputs $X \in \mathbb{R}^{N \times C}$; (5) solve the ridge regression

$$W_{\text{probe}} = (H^\top H + \lambda I)^{-1} H^\top X, \qquad \lambda = 10^{-3},$$

in closed form on a probe-train split; (6) evaluate held-out R^2 per channel on a probe-validation split: $R^2_k = 1 - \text{SS}_{\text{res},k}/\text{SS}_{\text{tot},k}$.

We probe two layers: layer 0 (the encoder output, i.e. the input to the first transformer block) and layer 3 (the output of the third and final transformer block, before the LayerNorm and the categorical head). $R^2 = 1$ at layer 0 confirms the encoder writes the raw channel into a linearly recoverable subspace; R^2 at layer 3 reveals how much of that recoverability survives the depth of the model. CI and CAT are excluded because their hidden states have a fundamentally different shape (per-channel stream and per-token respectively), so a single $H \to X$ probe does not apply.

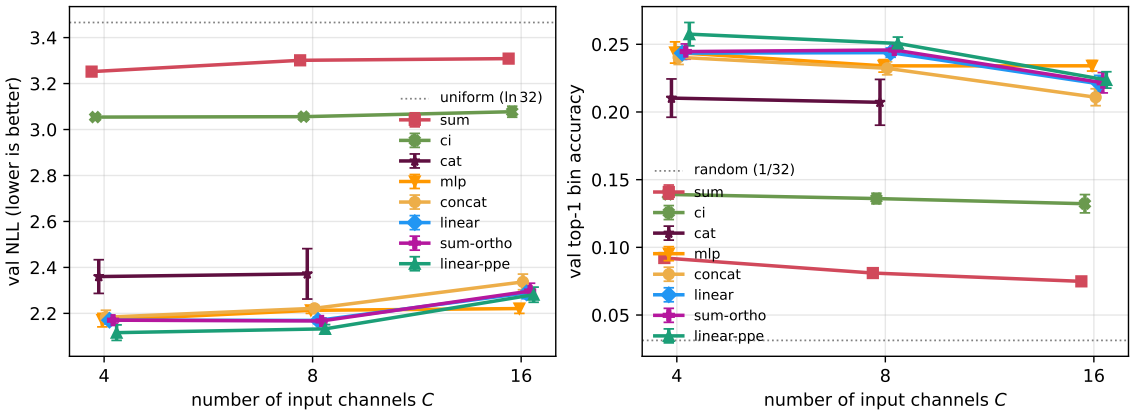

Figure 2: Scaling with input channel count C (error bars: ± 1 standard deviation across 20 seeds). CAT skipped at C=16 due to $\mathcal{O}((CT)^2)$ attention cost.

3 Results

3.1 Main comparison

Table 1 reports all eight encoders at C=4, d_{model}=64, 300 epochs with cosine LR decay, 20 seeds. The headline read is *practical near-equivalence among the per-channel-W_k family*: LINEAR, LINEAR-ORTHO, CONCAT, and MLP all fall in a 0.02 NLL band at 2.155–2.177, with LINEAR and LINEAR-ORTHO matching to three decimals (the orthogonality penalty contributes nothing at convergence). LINEAR-PPE sits a small step below at 2.114, a $\sim$0.04 NLL improvement over LINEAR that survives paired analysis at 20 seeds; the mechanism is a learned rotation of the positional encoding out of the channel subspace, established in Section 3.10. The gap corresponds to about 2% of the baseline NLL and is well within the spread that small task-design choices would induce; we read it as a real effect, not a practically transformative one. Below the top tier, the architectural alternatives underperform despite nominally giving each channel more capacity. CAT sits $\sim$0.18 NLL below the per-channel-W_k family at 2.348. CI drops decisively further at 3.053, only $\sim$0.2 NLL above the SUM floor of 3.257 and almost 0.9 NLL below the top tier; it is closer to the shared-scalar information-theoretic floor than to the linear baseline. So two encoders lose decisively on the synthetic benchmark: SUM, for the algebraic reasons discussed in the Discussion, and CI, whose head-side bottleneck and universal overfitting we unpack alongside the other architectural baseline in the Discussion.

3.2 Channel-count scaling

All encoders degrade as C grows (channels $k \geq 4$ are distractors). The per-channel-W_k family stays clustered at the top across $C \in \{4, 8, 16\}$ and the gap to SUM widens rather than narrows. LINEAR-PPE sits at the very top at C=4 and C=8 and ties MLP at C=16 (with MLP taking the lowest mean at that C); the paired-test statistics, and the geometric mechanism explaining why the LINEAR-PPE edge shrinks with C, are in Section 3.10.

At C=16, MLP pulls ahead of the linear family (2.231 vs. 2.27–2.34). Paired tests against LINEAR (p=2×10^{-4}, 18/20), LINEAR-ORTHO (p=7×10^{-4}, 17/20), and CONCAT (p=5×10^{-12}, 20/20) are decisive; against LINEAR-PPE the gap is small and the call is borderline ($\Delta = 0.020$, paired p=0.053, 12/20). Combined with the LINEAR-PPE lead at smaller C, the picture is a narrow top tier whose member ordering shifts with C: LINEAR-PPE edges the linear family at every C, MLP edges the

Table 2: Scaling with channel count. Channels 0–3 drive the outcome; additional channels are independent distractors. 300 epochs with cosine decay, mean ± std over 20 seeds. CAT is skipped at C=16 because its $\mathcal{O}((CT)^2)$ attention cost explodes. Sub-band sub-tables span the within-cell standard error times a t-quantile; encoder-vs-encoder paired tests reported in the text use the same paired-seed design across encoders.

C	encoder	val NLL ↓	val acc ↑
4	SUM	3.257 ± 0.011	0.091 ± 0.003
	CI	3.053 ± 0.013	0.136 ± 0.004
	CAT	2.348 ± 0.060	0.212 ± 0.011
	CONCAT	2.170 ± 0.025	0.243 ± 0.006
	MLP	2.177 ± 0.022	0.242 ± 0.007
	LINEAR-ORTHO	2.155 ± 0.020	0.247 ± 0.006
	LINEAR	2.155 ± 0.019	0.247 ± 0.006
	LINEAR-PPE	2.114 ± 0.029	0.256 ± 0.009
8	SUM	3.300 ± 0.006	0.081 ± 0.004
	CI	3.062 ± 0.016	0.135 ± 0.004
	CAT	2.334 ± 0.073	0.213 ± 0.011
	CONCAT	2.234 ± 0.023	0.231 ± 0.005
	MLP	2.211 ± 0.027	0.236 ± 0.005
	LINEAR-ORTHO	2.176 ± 0.022	0.244 ± 0.006
	LINEAR	2.176 ± 0.022	0.243 ± 0.006
	LINEAR-PPE	2.150 ± 0.030	0.249 ± 0.006
16	SUM	3.310 ± 0.004	0.075 ± 0.003
	CI	3.078 ± 0.017	0.133 ± 0.005
	CONCAT	2.340 ± 0.033	0.215 ± 0.008
	LINEAR-ORTHO	2.267 ± 0.038	0.227 ± 0.007
	LINEAR	2.267 ± 0.032	0.226 ± 0.007
	LINEAR-PPE	2.252 ± 0.038	0.230 ± 0.008
	MLP	2.231 ± 0.032	0.234 ± 0.004

linear family (but ties LINEAR-PPE) at C=16, and at every C the absolute spread among the top four is on the order of 0.02–0.05 NLL on a baseline of ∼2.2 NLL.

3.3 Data-richness check: C=16 at 10× training data

The ΔNLL ∼0.07 between MLP and LINEAR at C=16, N_{series}=512 raises an obvious question: is the nonlinear stem genuinely useful, or is the main sweep data-limited and MLP just happens to extract more from sparse data? We test this directly by retraining the same top-tier encoders at C=16 with N_{series}=5120 (10× the main-sweep data), 20 seeds, otherwise identical.

Two observations follow. MLP's lead survives in the data-rich regime: 18–20 of 20 paired-seed differences favour MLP over each of LINEAR (paired p=4×10^{-4}), LINEAR-ORTHO (p=9.6×10^{-4}), LINEAR-PPE (p=2.4×10^{-4}), and CONCAT (p=2.3×10^{-13}), so the C=16 MLP finding is not a data-limited artefact. But the magnitude shrinks roughly three-fold: ΔNLL(MLP–LINEAR) goes from 0.036 at N=512 to 0.011 at N=5120; the full linear family at N=5120 falls inside a band of width 0.024 NLL versus 0.073 at N=512. Each encoder also gains ∼0.4 NLL from the extra data, confirming

Table 3: C=16 top-tier encoders, N_{series}=5120 vs. N_{series}=512 (20 seeds each, 300 epochs). The full linear family at N=5120 falls inside a 0.024 NLL band.

encoder	N=512 val NLL	N=5120 val NLL	Δ from data
MLP	2.231 ± 0.032	1.831 ± 0.009	-0.400
LINEAR-PPE	2.252 ± 0.038	1.841 ± 0.013	-0.411
LINEAR	2.267 ± 0.032	1.843 ± 0.015	-0.424
LINEAR-ORTHO	2.267 ± 0.038	1.844 ± 0.018	-0.423
CONCAT	2.340 ± 0.033	1.854 ± 0.011	-0.486

the N=512 sweep is data-limited at C=16 just as at C=4 (Section 3.6). The encoder ordering at C=16 is robust to data scale; the absolute size of the gaps is not.

3.4 Model-width scaling

Table 4 varies d_{model} over $\{64, 128, 256\}$ at C=4, 20 seeds, all top-tier encoders plus the SUM and CONCAT baselines. Three observations.

First, SUM's ceiling is structural and capacity-independent: 15× more parameters move NLL by less than 0.01. The encoder destroys $(C-1)$ channels' worth of information through its shared scalar projection, and downstream capacity cannot recover what the encoder threw away (formal derivation in the Discussion).

Second, *every* encoder shows clear overfitting at $d_{\text{model}} \geq 128$. Best val NLL is reached at epoch ∼100 (d=128) or ∼40 (d=256), after which val NLL drifts upward (catastrophically for MLP at d=256, where the end-of-training val NLL exceeds $\ln 32$). Effective early stopping (Section 2.3) is what protects the reported best-NLL numbers; the trajectories themselves are not converged. C=4 with N_{series}=512 does not justify a 256-dim residual stream.

Third, the linear family's best-val ceiling rises gently with d_{model} ($2.16 \to 2.23 \to 2.32$ for LINEAR), indicating that the optimal effective-stopping model gets worse as the model is given more capacity to overfit. MLP degrades fastest of the top tier ($2.18 \to 2.30 \to 2.41$) because its encoder parameters scale as d_{model}^2 and exacerbate the capacity mismatch. The paired tests against LINEAR sharpen with width: p=2×10^{-4} at d=64 (18/20 favour LINEAR), p=4×10^{-9} at d=128 (20/20), and p=5×10^{-9} at d=256 (19/20). The C=4 small-channel regime does not reward nonlinearity, and gives it room to overfit when widened. This is in sharp contrast to C=16 where MLP leads (Section 3.2): the high-channel-pressure regime is what makes nonlinearity useful.

LINEAR-PPE also leads at every d_{model} tested, with paired gaps to LINEAR growing mildly with width and 56 of 60 paired-seed comparisons across the three widths favouring it. Details and the rotation-based mechanism that explains why more residual-stream width helps are in Section 3.10.

3.5 Gram-matrix analysis: spontaneous orthogonality

Applying the off-diagonal cosine diagnostic from Section 2.5 to the trained W_k:

Without any penalty the learned W_k are already near-orthogonal (mean off-diagonal cosine 0.042, 95% bootstrap CI $[0.037, 0.046]$). The regulariser tightens this by another factor of ∼4 to 0.010 (95% CI $[0.008, 0.011]$) but does not improve downstream NLL. The task loss itself provides enough gradient pressure to separate the per-channel projections once they exist as free parameters.

Table 4: Scaling with d_{model} at C=4. d_{ff}=4 d_{model}, 4 attention heads (so d_{head} grows). Mean ± std over 20 seeds, 300 epochs with cosine decay. *All* encoders flag as overfitting at $d_{\text{model}} \geq 128$: best val NLL is reached well before epoch 300 (typical best epochs ∼100 at d=128, ∼40 at d=256), and val NLL drifts upward afterwards. The reported best-NLL numbers are the effective-early-stopping minima.

d_{model}	encoder	total params	val NLL ↓	val acc ↑
64	SUM	152,480	3.257 ± 0.011	0.091 ± 0.003
	CONCAT	152,288	2.170 ± 0.026	0.243 ± 0.006
	LINEAR	152,672	2.155 ± 0.019	0.248 ± 0.005
	LINEAR-ORTHO	152,672	2.155 ± 0.019	0.248 ± 0.005
	MLP	156,640	2.177 ± 0.023	0.241 ± 0.008
	LINEAR-PPE	156,832	2.115 ± 0.029	0.256 ± 0.009
128	SUM	599,840	3.260 ± 0.010	0.091 ± 0.003
	CONCAT	599,456	2.229 ± 0.031	0.235 ± 0.006
	LINEAR	600,224	2.234 ± 0.032	0.235 ± 0.006
	LINEAR-ORTHO	600,224	2.233 ± 0.032	0.235 ± 0.007
	MLP	616,352	2.301 ± 0.027	0.222 ± 0.007
	LINEAR-PPE	616,736	2.185 ± 0.032	0.245 ± 0.005
256	SUM	2,379,296	3.262 ± 0.011	0.091 ± 0.003
	CONCAT	2,378,528	2.311 ± 0.026	0.217 ± 0.004
	LINEAR	2,380,064	2.322 ± 0.033	0.217 ± 0.006
	LINEAR-ORTHO	2,380,064	2.323 ± 0.036	0.217 ± 0.006
	MLP	2,445,088	2.407 ± 0.023	0.210 ± 0.007
	LINEAR-PPE	2,445,856	2.265 ± 0.026	0.229 ± 0.006

Does the MLP stem do the same thing? The same diagnostic applies to MLP's first-layer weight $W^{(1)} \in \mathbb{R}^{d_{\text{hidden}} \times C}$: its columns are the per-channel input directions before the GELU nonlinearity, and their pairwise cosines measure how much the encoder separates channels at the linear-stem stage versus deferring that separation to GELU plus $W^{(2)}$. Training MLP at $C \in \{4, 8, 16\}$ with 5 seeds and applying the off-diagonal cosine diagnostic to $W^{(1)}$'s columns:

MLP's linear stem is also pushed toward near-orthogonality by the task loss, but consistently looser than the linear family's W_k: at C=4 its mean off-diagonal cosine is 1.8× LINEAR's (0.075 vs. 0.042); at C=16 the gap widens further and pairs of columns reach $|\cos| \approx 0.46$. The mechanism is straightforward: MLP can also separate channels through the GELU plus $W^{(2)}$, so the gradient pressure on $W^{(1)}$ alone is partially relieved. The qualitative finding survives—spontaneous channel orthogonalisation is a property of the task loss interacting with a per-channel input stem, not unique to a single-layer projection—but quantitatively the linear family achieves the cleanest geometry.

Bias-ablation note. LINEAR's forward pass $\mathbf{h}(t) = \sum_k (W_k v_k(t) + b_k) + \mathbf{p}(t)$ has a redundancy: the $C \cdot d_{\text{model}}$ per-channel biases sum to a single constant offset that adds to every embedding regardless of input or position, so they cannot affect the cosine geometry. Retraining LINEAR with the channel bias zeroed and frozen (LINEAR-NOBIAS) confirms this experimentally: val NLL stays within seed std (2.177 ± 0.013 for LINEAR-NOBIAS vs. 2.169 ± 0.014 for LINEAR, both at 5 seeds), and mean/max $|\cos|$ match to three decimals. The LINEAR number here is a 5-seed sub-sweep run for the ablation and is slightly higher than the 20-seed main-table estimate (2.155 ± 0.019; both means

Table 5: Learned per-channel W_k geometry (C=4, 300 epochs, mean ± std over 20 seeds; same runs as Table 1).

encoder	best val NLL	$\max_{i \neq j} \lvert\cos(W_i, W_j)\rvert$	mean $\lvert\cos\rvert$
LINEAR	2.155 ± 0.019	0.095 ± 0.032	0.042 ± 0.010
LINEAR-ORTHO	2.155 ± 0.020	0.022 ± 0.008	0.010 ± 0.003

Table 6: MLP first-layer column geometry (300 epochs, d_{model}=64). This is a 5-seed sub-sweep run for the gram diagnostic, so the NLL column is noisier than the 20-seed main-table estimate (MLP at C=4 in Table 1 is 2.177 ± 0.022); the two estimates are within each other's seed std. Compare to LINEAR's W_k at C=4: mean 0.042, max 0.095.

C	best val NLL	mean $\lvert\cos\rvert$	max $\lvert\cos\rvert$
4	2.170 ± 0.036	0.075 ± 0.021	0.158 ± 0.058
8	2.214 ± 0.016	0.074 ± 0.015	0.248 ± 0.098
16	2.222 ± 0.025	0.105 ± 0.005	0.463 ± 0.109

within the same seed std). We retain the bias in LINEAR's headline definition to match the PyTorch `nn.Linear` default; the point of the ablation is that the geometric argument is bias-independent.

3.6 Encoding space allocation

Beyond separating channels, does the model allocate more encoding space to channels that matter more? Using the norm and variance-fraction diagnostics from Section 2.5. At C=4 every channel is a driver, so the driver/distractor split is trivial; we report $C \in \{8, 16\}$ where the question becomes meaningful.

Table 7: Encoding space allocation for LINEAR (300 epochs, mean over 20 seeds). Channels 0–3 always drive the outcome; the remaining ($C-4$) channels are distractors.

	$\lVert W_k \rVert$ norm		variance fraction	
	drivers (0–3)	distractors	drivers	distractors
C=8	1.22	0.54	83.8%	16.2%
C=16	1.30	0.59	62.4%	37.6%

Driver channels receive ∼2.2× larger seed-averaged norms than distractors at C=8 and ∼2.1× at C=16. The four drivers capture 83% of the embedding variance at C=8 and 61% at C=16, where the aggregate share is naturally diluted by having twelve distractors but the per-channel variance contribution remains markedly larger for drivers (∼15% each vs. ∼3% each). We do not report per-channel error bars on the norms, and the fine-grained ordering within drivers is descriptive only; the robust claim is the driver/distractor gap, which holds at both channel counts.

Why distractor norms are not zero. The gap is large but the distractor norms (∼0.55 at C=8) do not vanish. This is a finite-data equilibrium: the expected gradient on a distractor weight factorises to zero by the distractor's independence from the outcome, but stochastic estimates have $\mathcal{O}(1/\sqrt{N_{\text{series}}})$ variance, and under L2 weight decay distractor weights sit at the corresponding

Table 8: Linear-probe channel recovery at C=4, seed 0, 300 epochs. Closed-form ridge probe $\mathbf{h} \mapsto \hat{x}$ from a frozen hidden state to the raw input $x \in \mathbb{R}^4$; held-out per-channel R^2. Layer 0 is the input embedding; layer 3 is the final transformer block. All per-channel-W_k encoders achieve near-perfect input recovery and preserve mean $R^2 \geq 0.84$ through three attention layers; SUM collapses to $R^2 \approx 0.24$ at every depth.

Encoder	Layer	R^2_{ch0}	R^2_{ch1}	R^2_{ch2}	R^2_{ch3}	mean
SUM	0 (input)	0.255	0.250	0.242	0.214	0.240
SUM	3 (final)	0.248	0.242	0.235	0.206	0.233
LINEAR	0 (input)	1.000	1.000	1.000	1.000	1.000
LINEAR	3 (final)	0.957	0.861	0.911	0.879	0.902
LINEAR-ORTHO	0 (input)	1.000	1.000	1.000	1.000	1.000
LINEAR-ORTHO	3 (final)	0.959	0.856	0.915	0.885	0.904
MLP	0 (input)	1.000	1.000	1.000	1.000	1.000
MLP	3 (final)	0.934	0.879	0.913	0.646	0.843
LINEAR-PPE	0 (input)	1.000	1.000	1.000	1.000	1.000
LINEAR-PPE	3 (final)	0.930	0.861	0.874	0.847	0.878
CONCAT	0 (input)	0.998	1.000	1.000	1.000	0.999
CONCAT	3 (final)	0.955	0.852	0.906	0.845	0.890

noise floor rather than being driven to zero. The $1/\sqrt{N}$ scaling predicts a $\sim\sqrt{10} \approx 3.2$ reduction in distractor norms at N=5120 (the data-rich condition of Section 3.3); the observed reduction is $\sim$2.5×, close to but slightly slower than predicted (driver/distractor norm ratio rises from 2.2× to 6.7×, driver variance share from 83% to 98%).

3.7 Channel identifiability via linear probing

Following the procedure in Section 2.5, Table 8 reports per-channel R^2 at layer 0 (the encoder output) and layer 3 (the output of the final transformer block).

First, every per-channel-W_k encoder recovers the raw inputs near-perfectly at layer 0: LINEAR, LINEAR-ORTHO, LINEAR-PPE, and MLP all hit R^2=1.000 across all four channels, and CONCAT does the same up to rounding (R^2_{ch0}=0.998). The encoder is writing the channels into a linearly recoverable subspace by construction.

Second, deep recoverability survives the full three-block stack: mean R^2 at layer 3 stays at 0.84–0.90 for every per-channel-W_k encoder. MLP drops to 0.646 on ch3 while keeping the other three above 0.87; the other encoders degrade roughly uniformly. The result confirms the residual-stream prediction in Section 2.5: the linear channel directions the encoder writes at layer 0 persist through every block.

Third, SUM collapses to $R^2 \approx 0.24$ at every depth. This is essentially the information-theoretic floor $1/C = 0.25$ for recovering one of four independent unit-variance channels from their pointwise sum: the information loss happens at the encoder and is unrecoverable downstream.

3.8 Channel masking

The outcome formula gives us a known importance ordering: ch0 appears in two terms, interacting with ch1 and gated by ch3; ch1 appears once (interacting with ch0); ch2 enters alone as a sine; ch3 enters as a sign indicator on ch0. So we know a priori which channels the model *should* rely on most.

Table 9: Test-time channel ablation at C=4, seed 0, 300 epochs. We zero a single channel's input at inference; rows show the resulting val accuracy. Δ is accuracy drop from the unmasked baseline. The outcome formula uses ch0 in two interaction terms, ch1 in one, and ch2/ch3 once each with smaller marginal effect; the per-channel-W_k encoders reproduce this ordering cleanly, whereas SUM, CI, and CAT are flatter.

encoder	no mask	mask ch0	mask ch1	mask ch2	mask ch3
SUM	0.093	0.082	0.089	0.084	0.094
Δ acc	—	−0.011	−0.004	−0.009	+0.001
CI	0.144	0.082	0.108	0.086	0.134
Δ acc	—	−0.062	−0.036	−0.058	−0.010
CAT	0.217	0.095	0.110	0.110	0.200
Δ acc	—	−0.122	−0.107	−0.107	−0.017
CONCAT	0.241	0.096	0.113	0.103	0.202
Δ acc	—	−0.145	−0.128	−0.138	−0.039
LINEAR	0.238	0.096	0.116	0.106	0.204
Δ acc	—	−0.142	−0.122	−0.132	−0.034
LINEAR-ORTHO	0.239	0.096	0.117	0.104	0.205
Δ acc	—	−0.143	−0.122	−0.135	−0.034
MLP	0.240	0.098	0.111	0.106	0.209
Δ acc	—	−0.142	−0.129	−0.134	−0.031
LINEAR-PPE	0.256	0.096	0.115	0.106	0.216
Δ acc	—	−0.160	−0.141	−0.150	−0.040

Zeroing one channel at test time gives us a behavioural test of whether the encoder reproduces this ordering. Table 9 shows that the top-tier encoders produce sharp, interpretable per-channel accuracy drops that match the formula's importance ordering. SUM is nearly flat, as expected from the information-theoretic argument: it has no recoverable channel identity for the masking to disturb.

3.9 Convergence and wall-clock cost

Two complementary worries about more involved encoders: (i) that they cost extra training time even when they don't change the ceiling, and (ii) the converse—that they might, despite tying or losing on best-NLL, at least *reach* that ceiling faster and therefore be the right choice when compute or epochs are constrained. From the 300-epoch traces (val NLL recorded every 20 epochs, C=4, 5 seeds) we extract two diagnostics: the epoch at which each run first reaches within 0.05 NLL of its own best, and the wall-clock seconds per epoch, normalised to LINEAR.

The five top-tier encoders reach within 0.05 NLL of their final value at the same trace point (∼150 epochs) and cost essentially the same per-epoch wall time—the transformer backbone dominates, so encoder choice within this family is free. Neither worry materialises: the more involved encoders do not slow convergence (the MLP stem's 9× extra parameters cost nothing per epoch and reach target on the same trace point), but they also do not converge faster than LINEAR. There is no "faster-to-target" argument available for picking MLP, LINEAR-ORTHO, or LINEAR-PPE over LINEAR within the top tier.

CI and CAT pay substantial wall-clock costs from their architectures, not their input encoders: ∼2.3× LINEAR at C=4 for CI (scaling to ∼8× at C=16) and ∼5× at C=4 for CAT (scaling with C^2T^2

Table 10: Convergence speed and wall-clock cost (C=4, 300 epochs, 5 seeds, single GPU). Epochs-to-target is rounded to the trace resolution of 20 epochs. Cost is per-epoch wall time normalised to LINEAR.

encoder	best NLL	epoch to NLL +0.05	relative cost
LINEAR	2.170	∼152	1.00×
LINEAR-ORTHO	2.170	∼156	1.04×
MLP	2.171	∼144	1.01×
CONCAT	2.183	∼152	1.00×
LINEAR-PPE	2.116	∼148	0.99×
SUM	3.252	∼20 (plateau)	1.00×
CI	3.054	∼20 (plateau)	2.3×
CAT	2.360	∼180	5.2×

to ∼17× at C=8). CAT also takes longer (∼180 epochs vs. ∼150) to reach its own—worse—best NLL on the synthetic benchmark.

3.10 Positional projection: the orthogonalisation mechanism

LINEAR-PPE achieves the lowest val NLL at C=4 and C=8 and ties MLP at C=16 (the two are within each other's seed std at C=16, where MLP has the lower mean 2.231 vs. LINEAR-PPE's 2.252). Within the linear family, LINEAR-PPE edges LINEAR at every C. The paired-seed design (Section 2.3) gives one Δ_s per seed; at 20 seeds the LINEAR-PPE vs. LINEAR picture is:

- C=4: $\Delta = 0.041$, paired t=6.60, p=2.6×10^{-6}, 95% bootstrap CI $[+0.029, +0.053]$, 19/20 favour ppe.
- C=8: $\Delta = 0.026$, paired p=3.7×10^{-4}, 95% bootstrap CI $[+0.014, +0.038]$, 16/20.
- C=16: $\Delta = 0.016$, paired p=0.036, 95% bootstrap CI $[+0.002, +0.028]$, 14/20.

The gap shrinks with C but does not vanish; even at C=16, where the earlier 5-seed analysis read as null, the 20-seed paired test puts Δ above zero, just barely. The advantage is small in magnitude—∼2% of baseline NLL at C=4, ∼0.7% at C=16—and the practical-significance reading is more subdued than the statistical-significance one would suggest in isolation.

Direct measurement: orthogonalisation, not compression. There are two natural mechanisms by which a learned linear projection on top of $\mathbf{p}(t)$ could help. The *orthogonalisation* reading: W_{pos} rotates the positional basis out of span(W), so position and channels stop competing for the same coordinates of the residual stream. The *compression* reading: W_{pos} projects $\mathbf{p}(t)$ onto a small effective subspace, leaving the remaining dimensions free for channels. The two predictions differ: orthogonalisation should reduce the overlap between the positional basis and span(W) while leaving the basis's intrinsic rank essentially unchanged; compression should reduce that rank. We test both directly.

We extract the per-channel projection matrix $W \in \mathbb{R}^{C \times d_{\text{model}}}$ and the effective positional basis $P \in \mathbb{R}^{T \times d_{\text{model}}}$ from trained LINEAR and LINEAR-PPE models (C=4, 20 seeds each). For LINEAR, $P_{t,:} = \mathbf{p}(t)$ is the fixed sinusoidal basis; for LINEAR-PPE, $P_{t,:} = W_{\text{pos}}\mathbf{p}(t) + \mathbf{b}_{\text{pos}}$ is the learned-rotated basis. Two metrics suffice to discriminate the mechanism:

Table 11: Geometric measurement of the positional basis P and its overlap with the channel subspace span(W) (C=4, 300 epochs, mean over 20 seeds with 95% bootstrap CIs in brackets where seed variance is non-zero).

	LINEAR	LINEAR-PPE
Effective rank of P (entropy of $\tilde{\sigma}^2$)	7.59	9.55 $[9.17, 9.97]$
Fraction of $\|P\|_F^2$ within span(W)	0.034 $[0.029, 0.039]$	0.005 $[0.005, 0.006]$

The two metrics give opposite verdicts on the two readings. Orthogonalisation is confirmed: the fraction of P's energy that lies inside span(W) drops by $\sim$6.3$\times$ under LINEAR-PPE (3.4% $\rightarrow$ 0.5%; paired-difference 95% bootstrap CI on the reduction $[-0.034, -0.024]$), so the learned W_{pos} rotation actively pushes the positional encoding out of the channel subspace. Compression is contradicted: P's effective rank is *higher* under LINEAR-PPE (9.55 vs. 7.59), not lower—the rotation spreads positional energy across more directions, not fewer. The mechanism is rotation, not compression.

The redistribution is visible at the level of individual singular values: LINEAR's P has one dominant direction ($\sigma_0 \approx 52$) plus a long tail; LINEAR-PPE's is more evenly distributed ($\sigma_0 \approx 25$, with the next several around 11–13). Rotating the dominant positional mode out of span(W) inevitably redistributes its energy across other directions, which is the same effect read either as "orthogonalising position from channels" or as "flattening the singular spectrum of P".

(Principal angles between span(W) and span(P) collapse to $\sim$0 at this dimensionality—span(W) is C-dimensional and trivially contained in span(P)'s $\sim$22-rank subspace—so they measure subset membership rather than energetic overlap and are not the right tool here.)

With orthogonalisation confirmed, the C-dependent shrinkage of the gap has a clean mechanistic reading: W_{pos} must rotate P out of a C-dimensional channel span using a fixed d_{model}^2 parameter budget, and the rotation becomes increasingly over-constrained as C grows. At C=4 there are $d_{\text{model}} - C = 60$ "free" dimensions for P to occupy; at C=16 only 48, with the channel subspace also pressing harder on the residual stream overall. The same mechanism explains why the lead grows mildly with d_{model} in the Section 3.4 sweep—more residual-stream capacity gives the rotation more directions to move into. Two further factors—only four channels are drivers regardless of C, and seed-level optimisation variance grows with C—could contribute, but neither directly invokes the rotation mechanism this section establishes.

3.11 Loss-family robustness check: MSE target

The categorical K=32-bin head could in principle interact with encoder choice—e.g. by rewarding encoders that resolve fine target quantiles—so we rerun a subset of the synthetic main sweep with a scalar regression head trained on the continuous (pre-binning) target under MSE loss. Same backbone, same training schedule, 5 seeds, $C \in \{4, 16\}$, and the same encoder set as the d_{model} sweep (SUM, CONCAT, LINEAR, LINEAR-ORTHO, MLP, LINEAR-PPE); see Table 12. The ordering is preserved at both channel counts: SUM collapses to $R^2 \approx 0.13$ at C=4 and ≈ 0.01 at C=16, consistent with the information-theoretic ceiling argument; the per-channel-W_k tier clusters tight; LINEAR-PPE edges LINEAR at C=4 ($\sim$6% relative reduction in MSE, 0.058 vs. 0.062), and MLP edges LINEAR at C=16. The categorical-vs.-MSE choice is therefore not what is driving the encoder ordering—both losses penalise SUM for the same information-theoretic reason, and both reveal the same small gaps inside the top tier. The MSE sweep is not powered (5 seeds, no paired design) to resolve the within-tier gaps at the statistical level of the categorical 20-seed analysis; what it does is rule out that the entire ordering is a label-binning artefact.

Table 12: Loss-family robustness check: MSE/regression target replaces categorical cross-entropy. Same backbone, 5 seeds, 300 epochs, d_{model}=64. Lower MSE is better. The ordering survives the swap: SUM collapses ($R^2 \approx 0.13$ at C=4, ≈ 0.01 at C=16), the per-channel-W_k tier (CONCAT, LINEAR, LINEAR-ORTHO, MLP, LINEAR-PPE) clusters tight, and LINEAR-PPE edges LINEAR at C=4 (∼6% MSE reduction). MLP pulls ahead of LINEAR at C=16, mirroring the categorical sweep.

	C=4		C=16	
encoder	val MSE ↓	val R^2 ↑	val MSE ↓	val R^2 ↑
SUM	0.4719 ± 0.0090	0.128 ± 0.016	0.5338 ± 0.0167	0.011 ± 0.006
CONCAT	0.0636 ± 0.0027	0.882 ± 0.006	0.0862 ± 0.0071	0.840 ± 0.014
LINEAR	0.0619 ± 0.0024	0.886 ± 0.005	0.0736 ± 0.0026	0.863 ± 0.008
LINEAR-ORTHO	0.0619 ± 0.0024	0.886 ± 0.005	0.0735 ± 0.0020	0.864 ± 0.007
MLP	0.0643 ± 0.0044	0.881 ± 0.009	0.0678 ± 0.0025	0.874 ± 0.007
LINEAR-PPE	0.0582 ± 0.0031	0.892 ± 0.007	0.0722 ± 0.0033	0.866 ± 0.007

Table 13: ETTh1 next-step bin prediction. 7 variates, T=160, K=32 quantile bins on the target OT (oil temperature) computed from the train split. Target is the next-step bin of OT; all 7 variates are inputs. d_{model}=56, 7 heads, 3 layers, d_{ff}=224; 300 epochs, cosine decay, mean ± std over 20 seeds (5 canonical + 15 paired-seed extras).

encoder	val NLL ↓	val acc ↑
SUM	3.668 ± 0.063	0.016 ± 0.020
CI	0.865 ± 0.038	0.664 ± 0.018
CAT	0.551 ± 0.019	0.785 ± 0.010
LINEAR	0.561 ± 0.017	0.786 ± 0.008
LINEAR-ORTHO	0.561 ± 0.017	0.784 ± 0.009
CONCAT	0.571 ± 0.019	0.783 ± 0.013
LINEAR-PPE	0.573 ± 0.014	0.776 ± 0.009
MLP	0.585 ± 0.020	0.788 ± 0.011

3.12 Real-data validation: ETTh1

Table 13 mostly confirms the synthetic ordering on ETTh1: the per-channel-W_k family (LINEAR, LINEAR-ORTHO, CONCAT, LINEAR-PPE, MLP) cluster within one standard deviation across seeds of one another, CI underperforms, and SUM fails catastrophically (NLL near $\ln 32 \approx 3.47$).

One difference from the synthetic ranking: CAT, which sits in the middle of the synthetic table, here posts the lowest mean NLL (0.551). Paired-difference tests at 20 seeds give a nuanced picture:

- Decisive: CAT beats MLP (paired p=1.4×10^{-7}, 20/20) and LINEAR-PPE (p=3×10^{-4}, 17/20).
- Marginal: CAT beats CONCAT (p=0.002, 16/20).
- Indistinguishable: CAT vs. LINEAR ($\Delta = 0.010$, paired p=0.14, 13/20) and CAT vs. LINEAR-ORTHO ($\Delta = 0.010$, paired p=0.10, 13/20).

Best-bin accuracies across the top tier are indistinguishable (0.78–0.79 with standard deviation across seeds ∼0.01). So CAT posts the lowest mean NLL but is statistically tied with LINEAR and

LINEAR-ORTHO at the seed level we test. The finding is not that channel-as-token decisively wins on real data; rather, its synthetic-benchmark disadvantage closes and a small directional lead emerges that does not survive a paired test against the closest competitors. A plausible reading is that ETTh1's seven variates are all informative measurements of the same underlying electrical system (oil temperature plus six grid-load variables)—unlike the synthetic benchmark, which deliberately mixes drivers with independent distractors—so the fine-grained cross-variate attention CAT enables has more to work with. Whether that pays *enough* to prefer it over the per-channel-W_k default at matched wall-clock budget is much less clear.

4 Discussion

linear is not new. Stacking per-channel projections W_k and summing is `nn.Linear(C, d_model)`—the most obvious default. Our contribution is not proposing it but auditing it: at matched parameter budget, it matches every alternative (block-partitioned, orthogonality-regularised, nonlinear, channel-independent, channel-as-token) on the synthetic benchmark, and ties at the top of the per-channel-W_k tier on ETTh1. Only the shared-projection SUM consistently loses.

What fails about the shared-scalar baseline. SUM collapses because, with a shared scalar projection W and constant per-channel embeddings, the encoder depends on its inputs only through their pointwise sum $S(t) = \sum_k v_k(t)$. That map has a $(C-1)$-dimensional null space, and by the data-processing inequality no downstream layer can recover what is destroyed before the transformer sees it. The probe makes the floor explicit: for independent unit-variance channels at C=4 the theoretical ceiling for linearly recovering one channel from S is $\rho^2 = 1/C = 0.25$, and the layer-0 probe of SUM sits at $R^2 \approx 0.24$, touching it (Table 8). The 15× d_{model} sweep in Table 4 moves NLL by less than 0.005 because the extra parameters never get to see the lost information. The bottleneck is not summation per se—LINEAR also sums—but the *shared* projection, which collapses the per-channel directions into one before the sum.

The residual stream carries channel identity end-to-end. Linear probes at layer 3 recover each channel to $R^2 \geq 0.84$, confirming the residual-stream prediction in Section 2.5: per-channel-W_k encoders write near-orthogonal channel directions into the residual stream at layer 0 and the pre-LN architecture preserves them through every block essentially by construction, since LayerNorm only rescales and the additive residual keeps them in the basis. This is the mechanistic-interpretability picture of the residual stream as an additive communication channel (4) applied at the level of input channels: the encoder chooses the subspace, the backbone passes it forward, and the deep-probe recoverability falls out of both.

Reading for non-numerical inputs. The mechanism is geometric and encoding-agnostic, which suggests one natural extension worth flagging: when the per-channel inputs are precomputed embeddings from an outside source (e.g. text, image, or tabular feature embeddings fed into a multi-stream transformer), the LINEAR-PPE-style learned rotation has a ready analog as a per-stream rotation that pushes precomputed embedding subspaces off the positional subspace and off one another. Whether this gives gains comparable to the ∼2% NLL improvement we measure here, or larger because external embeddings carry more structured competing geometry, is an interesting question for another study.

Statistical significance vs. practical near-equivalence. The 20-seed paired tests reported above are powerful enough to resolve several small inter-encoder gaps; it is worth separating that statistical power from a practical claim. Among the per-channel-W_k family (LINEAR, LINEAR-ORTHO, CONCAT, MLP, LINEAR-PPE) at C=4 the absolute NLL differences span 0.02 on a baseline of 2.16; LINEAR-PPE's lead over LINEAR is 0.041 NLL (∼2% of baseline); MLP's lead over the linear family at C=16 is 0.011–0.025 in the data-rich regime. These are real gaps under our experimental design, but they sit inside the spread that typical task-design choices (channel count, sequence length, label binning, etc.) would induce, and we would not expect them to translate cleanly to practically significant differences on new tasks with new data distributions. The practical recommendation that follows from this is stated in the Conclusion.

5 Limitations

- The synthetic benchmark deliberately makes channel identity informative. On tasks with exchangeable channels the encoder design problem itself changes—channel identity ceases to be a useful signal—so SUM is not penalised and the per-channel-W_k family's advantage over it should shrink or disappear entirely.
- Main sweep at d_{model}=64, $C \in \{4, 8, 16\}$; ETTh1 at d_{model}=56, C=7. The d_{model} sweep covers all top-tier encoders plus SUM at $d \in \{64, 128, 256\}, C$=4, but all encoders overfit at $d \geq 128$ under the main-sweep training data (Table 4); the reported best-NLL numbers there are effective-early-stopping minima rather than converged values.
- The main sweep at N_{series}=512 is data-limited. 10× training data improves all top-tier encoders by ∼0.4 NLL at C=16 (Table 3). The relative ordering of encoders is robust to data scale, but absolute inter-encoder gaps shrink ∼5× at 10× data; readers should treat the N=512 inter-encoder gaps as upper bounds on the data-rich-regime gaps.
- CAT is skipped at C=16 on the synthetic benchmark for compute reasons ($\mathcal{O}((CT)^2)$ attention).
- Headline comparisons are at 20 paired seeds. A few diagnostics (linear probing, channel masking, channel-bias ablation, MLP first-layer geometry, MSE loss-family check) remain at 5 seeds, labelled as such in their tables.

6 Related work

Channel-as-token and channel-independent TSF. iTransformer (7) treats each variate as a token; PatchTST (8) runs independent backbones per channel; Crossformer (13) does both. These preserve per-channel capacity at the cost of longer sequences or per-channel compute. Our finding is that on the synthetic benchmark a per-channel linear stem on a shared backbone (one token per time step, d_{model}-dim) matches both architectures at C=4, 8, 16; on ETTh1 CAT posts the lowest mean NLL but is statistically tied with LINEAR and LINEAR-ORTHO under paired analysis.

Linear and MLP stems in time-series forecasting. Many multivariate transformers use a linear input stem (10, 14), and a parallel line of work argues that the transformer machinery is itself often unnecessary: Zeng et al. (12) show that simple linear forecasters (DLINEAR, NLINEAR) match or beat several transformer architectures on standard multivariate benchmarks, and Chen et al. (3) make the same point with an all-MLP architecture (TSMIXER) that separates channel-mixing

from time-mixing. Our work is on the orthogonal question of how the input layer of a transformer should embed multiple channels rather than whether a transformer is needed at all; the two lines of evidence are compatible—a linear input projection is also a strong default—and our LINEAR baseline carries that lesson inside the encoder even when the backbone is non-linear. The relationship to TSMIXER is particularly direct: its channel-mixing MLP is essentially our MLP encoder used as the centrepiece rather than as an input stem.

Orthogonality regularisation. Bansal et al. (2) use soft-orthogonality penalties for feature decorrelation in deep networks. We apply the same idea to per-channel input projections and find it unnecessary: the task loss produces the same geometry.

Channel-importance weighting. Variable Selection Networks in the Temporal Fusion Transformer (6) explicitly learn per-channel importance weights via a gating mechanism on top of per-channel embeddings, on the hypothesis that channels need an architectural mechanism to be weighted by relevance. Our encoding-space-allocation result (Section 3.6) is partly negative evidence for that hypothesis in the synthetic setting: a bare `nn.Linear` discovers the driver/distractor split through its learned $\|W_k\|$ alone, with no explicit weighting mechanism. We do not claim this generalises to the harder settings TFT was designed for—static covariates, mixed categorical/continuous inputs, long-horizon multi-step forecasting—but at least for the homogeneous-numerical input case the explicit selection mechanism is solving a problem the encoder already solves.

Decoupling position from content. Ke et al. (5) showed that untying positional embeddings from content embeddings—routing position through a separate projection rather than additively combining them at the input— improves transformer language models. LINEAR-PPE applies the same principle to the channel–position competition in time-series transformers, and the direct geometric measurement in Section 3.10 identifies the mechanism as the same one: position is rotated out of the channel subspace so the two stop competing for the same residual-stream coordinates.

7 Conclusion

The encoder design problem turns out to be largely settled by the task loss itself. A per-channel linear projection (`nn.Linear(C,` $d_{\texttt{model}}$`)`) matches every more elaborate encoder we tested up to small, statistically real but practically modest, differences. The task loss drives the channel projections to near-orthogonality and scales their norms with channel importance; a shared-scalar projection has a capacity-independent information-theoretic ceiling. The exceptions resolved under paired analysis are small and qualified: LINEAR-PPE leads the linear family at every C by $\sim$2% NLL via positional–channel orthogonalisation, MLP edges them at C=16 with the gap shrinking $\sim$3$\times$ under 10$\times$ training data, and CAT posts the lowest mean NLL on ETTh1 but is statistically tied with LINEAR and LINEAR-ORTHO.

Practical recommendation. Default to `nn.Linear(C,` $d_{\texttt{model}}$`)`: within the per-channel-W_k family the encoder choice is, for any practical purpose, close to a free parameter, and the simplest option is the path of least surprise. LINEAR-PPE is a justified opportunistic extra when d^2_{model} parameters are cheap relative to the backbone. Reach for MLP or CAT only when the task gives a real reason—a high-channel regime where nonlinear gating may help, or a setting whose cross-variate structure justifies CAT's $\sim$5$\times$ wall-clock cost and $C{\cdot}T$ context length.

When we recognise, that a per channel linear projection is all that is needed for serving the input signals to the main model as orthogonally distinct entities for it to work from, we can focus

our modelling efforts and complexity to where it matters.

References

[1] G. Alain and Y. Bengio. Understanding intermediate layers using linear classifier probes. In *ICLR Workshop*, 2017.

[2] N. Bansal, X. Chen, and Z. Wang. Can we gain more from orthogonality regularizations in training deep networks? In *NeurIPS*, 2018.

[3] S.-A. Chen, C.-L. Li, N. Yoder, S. O. Arık, and T. Pfister. TSMixer: an all-MLP architecture for time series forecasting. *Transactions on Machine Learning Research*, 2023.

[4] N. Elhage, N. Nanda, C. Olsson, T. Henighan, N. Joseph, B. Mann, A. Askell, Y. Bai, A. Chen, T. Conerly, N. DasSarma, D. Drain, D. Ganguli, Z. Hatfield-Dodds, D. Hernandez, A. Jones, J. Kernion, L. Lovitt, K. Ndousse, D. Amodei, T. Brown, J. Clark, J. Kaplan, S. McCandlish, and C. Olah. A mathematical framework for transformer circuits. *Transformer Circuits Thread*, 2021.

[5] G. Ke, D. He, and T.-Y. Liu. Rethinking positional encoding in language pre-training. In *ICLR*, 2021.

[6] B. Lim, S. Ö. Arık, N. Loeff, and T. Pfister. Temporal fusion transformers for interpretable multi-horizon time series forecasting. *International Journal of Forecasting*, 37(4):1748–1764, 2021.

[7] Y. Liu, T. Hu, H. Zhang, H. Wu, S. Wang, L. Ma, and M. Long. iTransformer: inverted transformers are effective for time series forecasting. In *ICLR*, 2024.

[8] Y. Nie, N. H. Nguyen, P. Sinthong, and J. Kalagnanam. A time series is worth 64 words: long-term forecasting with transformers. In *ICLR*, 2023.

[9] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. In *NeurIPS*, 2017.

[10] H. Wu, T. Hu, Y. Liu, H. Zhou, J. Wang, and M. Long. TimesNet: temporal 2D-variation modeling for general time series analysis. In *ICLR*, 2023.

[11] R. Xiong, Y. Yang, D. He, K. Zheng, S. Zheng, C. Xing, H. Zhang, Y. Lan, L. Wang, and T.-Y. Liu. On layer normalization in the transformer architecture. In *ICML*, 2020.

[12] A. Zeng, M. Chen, L. Zhang, and Q. Xu. Are transformers effective for time series forecasting? In *AAAI*, 2023.

[13] Y. Zhang and J. Yan. Crossformer: transformer utilizing cross-dimension dependency for multivariate time series forecasting. In *ICLR*, 2023.

[14] H. Zhou, S. Zhang, J. Peng, S. Zhang, J. Li, H. Xiong, and W. Zhang. Informer: beyond efficient transformer for long sequence time-series forecasting. In *AAAI*, 2021.